In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

import warnings
warnings.filterwarnings("ignore")

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

In [2]:
```
df = pd.read_csv("car data.csv")

print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)

display(df.head())
```

```
Dataset loaded successfully!
Dataset shape: (301, 9)
```

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual	0
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual	0
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual	0
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual	0
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual	0

In [3]:
```
# Dataset information
print("Dataset Information:")
df.info()

print("\nStatistical Summary:")
display(df.describe(include="all"))

print("\nColumn Names:")
print(df.columns.tolist())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Records:", df.duplicated().sum())

print("\nData Types:")
print(df.dtypes)
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 301 entries, 0 to 300
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Car_Name       301 non-null    object
 1   Year           301 non-null    int64
 2   Selling_Price  301 non-null    float64
 3   Present_Price  301 non-null    float64
 4   Kms_Driven     301 non-null    int64
 5   Fuel_Type      301 non-null    object
 6   Seller_Type    301 non-null    object
 7   Transmission   301 non-null    object
 8   Owner          301 non-null    int64
dtypes: float64(2), int64(3), object(4)
memory usage: 21.3+ KB
```

Statistical Summary:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission
count	301	301.000000	301.000000	301.000000	301.000000	301	301	301
unique	98	NaN	NaN	NaN	NaN	3	2	2
top	city	NaN	NaN	NaN	NaN	Petrol	Dealer	Manual
freq	26	NaN	NaN	NaN	NaN	239	195	261
mean	NaN	2013.627907	4.661296	7.628472	36947.205980	NaN	NaN	NaN
std	NaN	2.891554	5.082812	8.644115	38886.883882	NaN	NaN	NaN
min	NaN	2003.000000	0.100000	0.320000	500.000000	NaN	NaN	NaN
25%	NaN	2012.000000	0.900000	1.200000	15000.000000	NaN	NaN	NaN
50%	NaN	2014.000000	3.600000	6.400000	32000.000000	NaN	NaN	NaN
75%	NaN	2016.000000	6.000000	9.900000	48767.000000	NaN	NaN	NaN
max	NaN	2018.000000	35.000000	92.600000	500000.000000	NaN	NaN	NaN

```
Column Names:
['Car_Name', 'Year', 'Selling_Price', 'Present_Price', 'Kms_Driven', 'Fuel_Type', 'Seller_Type', 'Transmission',
'Owner']

Missing Values:
Car_Name         0
Year             0
Selling_Price    0
Present_Price    0
Kms_Driven       0
Fuel_Type        0
Seller_Type      0
Transmission     0
Owner            0
dtype: int64

Duplicate Records: 2

Data Types:
Car_Name          object
Year               int64
Selling_Price    float64
Present_Price    float64
Kms_Driven         int64
Fuel_Type         object
Seller_Type       object
Transmission      object
Owner              int64
dtype: object
```

In [4]:

```python
# Remove duplicate records
df = df.drop_duplicates().copy()

# Remove unnecessary column
df = df.drop(columns=["Car_Name"], errors="ignore")

# Check missing values again
print("Missing values after removing duplicates:")
print(df.isnull().sum())

# Detect outliers using the IQR method
numeric_cols = df.select_dtypes(include=np.number).columns
```

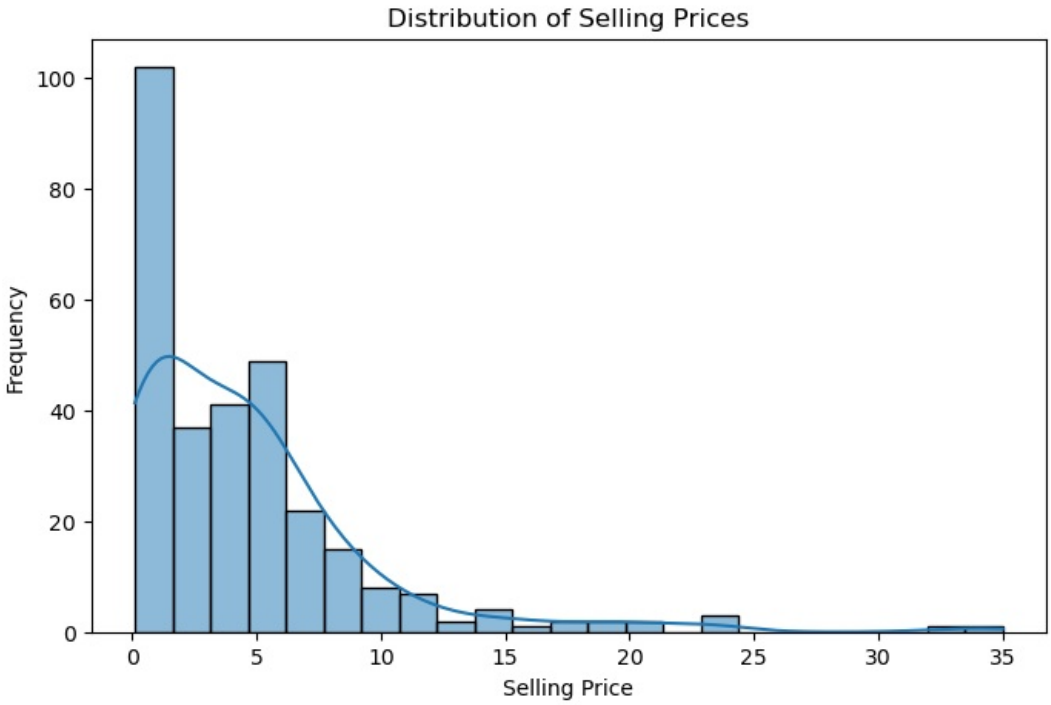
```
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

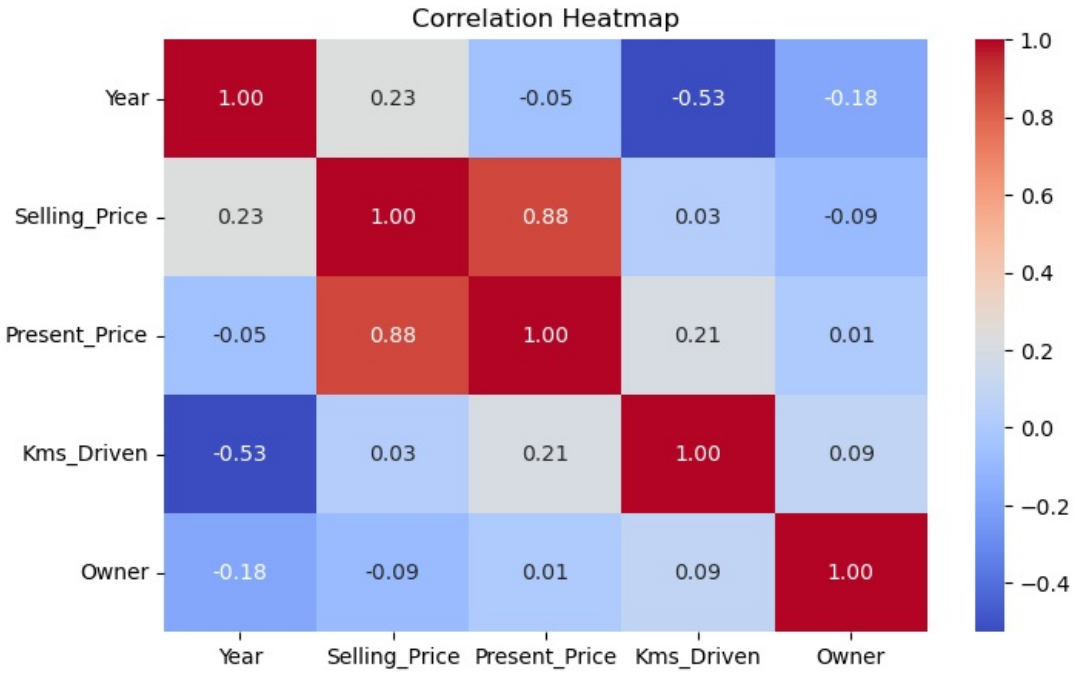
    count = ((df[col] < lower) | (df[col] > upper)).sum()
    print(f"{col}: {count} potential outliers")
```

```
Missing values after removing duplicates:
Year             0
Selling_Price    0
Present_Price    0
Kms_Driven       0
Fuel_Type        0
Seller_Type      0
Transmission     0
Owner            0
dtype: int64
Year: 7 potential outliers
Selling_Price: 16 potential outliers
Present_Price: 14 potential outliers
Kms_Driven: 8 potential outliers
Owner: 11 potential outliers
```

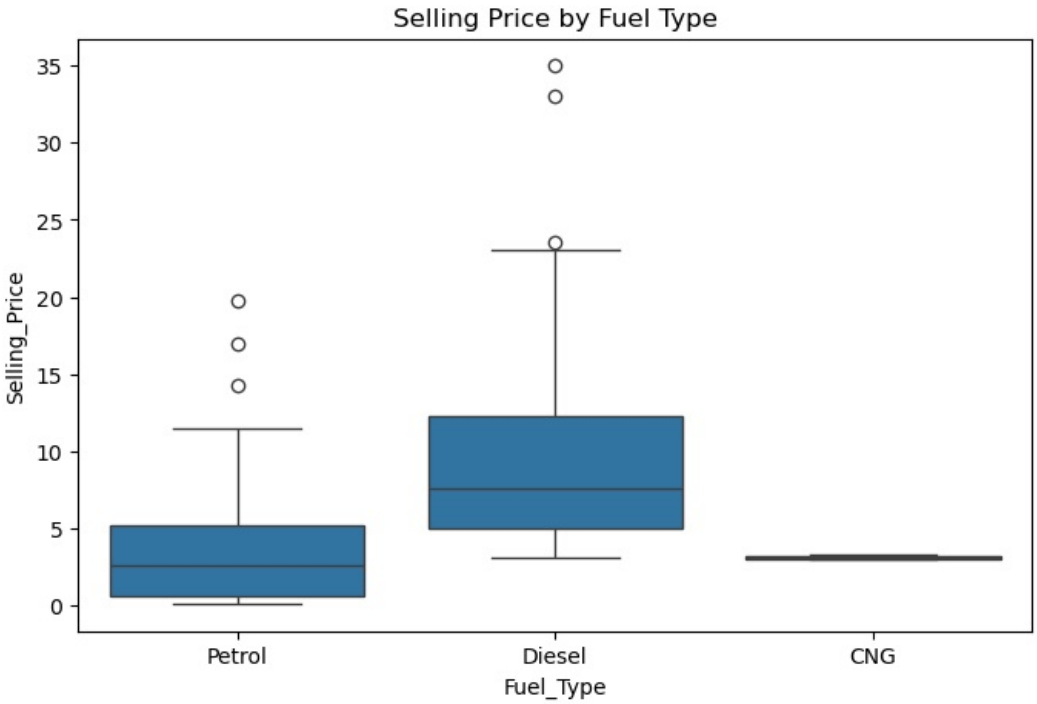
In [7]:
```
plt.figure(figsize=(8, 5))
sns.histplot(df["Selling_Price"], kde=True)
plt.title("Distribution of Selling Prices")
plt.xlabel("Selling Price")
plt.ylabel("Frequency")
plt.show()
```



In [8]:
```
plt.figure(figsize=(8, 5))
sns.heatmap(
    df.select_dtypes(include=np.number).corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title("Correlation Heatmap")
plt.show()
```



In [10]:
```
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x="Fuel_Type", y="Selling_Price")
plt.title("Selling Price by Fuel Type")
plt.show()
```



In [11]:
```
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=df,
    x="Present_Price",
    y="Selling_Price"
)
plt.title("Present Price vs Selling Price")
plt.show()
```



In [12]:
```
X = df.drop(columns=["Selling_Price"])
y = df["Selling_Price"]
print("Input features:")
print(X.columns.tolist())
print("\nTarget variable: Selling_Price")
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
Input features:
['Year', 'Present_Price', 'Kms_Driven', 'Fuel_Type', 'Seller_Type', 'Transmission', 'Owner']

Target variable: Selling_Price
X shape: (299, 7)
y shape: (299,)
```

In [13]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
print("Training records:", X_train.shape[0])
print("Testing records:", X_test.shape[0])
```

```
Training records: 239
Testing records: 60
```

In [14]:
```
numeric_features = X.select_dtypes(
    include=np.number
).columns.tolist()

categorical_features = X.select_dtypes(
    exclude=np.number
).columns.tolist()
```

In [15]:
```
linear_preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            Pipeline([
                ("imputer", SimpleImputer(strategy="median")),
                ("scaler", StandardScaler())
            ]),
            numeric_features
        ),
        (
            "cat",
            Pipeline([
                ("imputer", SimpleImputer(strategy="most_frequent")),
                ("encoder", OneHotEncoder(handle_unknown="ignore"))
            ]),
            categorical_features
        )
    ]
)
```

In [16]:
```
tree_preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            SimpleImputer(strategy="median"),
            numeric_features
        ),
        (
            "cat",
            Pipeline([
                ("imputer", SimpleImputer(strategy="most_frequent")),
                ("encoder", OneHotEncoder(handle_unknown="ignore"))
            ]),
            categorical_features
        )
    ]
)

print("Categorical encoding and preprocessing configured successfully!")
```

```
Categorical encoding and preprocessing configured successfully!
```

In [17]:
```
models = {
    "Linear Regression": Pipeline([
        ("preprocessor", linear_preprocessor),
        ("model", LinearRegression())
    ]),

    "Decision Tree Regressor": Pipeline([
        ("preprocessor", tree_preprocessor),
        ("model", DecisionTreeRegressor(random_state=42))
    ]),

    "Random Forest Regressor": Pipeline([
        ("preprocessor", tree_preprocessor),
        ("model", RandomForestRegressor(
            n_estimators=200,
            random_state=42,
            n_jobs=-1
        ))
    ])
}
```

In [18]:
```
for name, model in models.items():
    model.fit(X_train, y_train)
    print(name, "trained successfully!")
```

```
Linear Regression trained successfully!
Decision Tree Regressor trained successfully!
Random Forest Regressor trained successfully!
```

In [19]:
```
results = []
predictions = {}
for name, model in models.items():
    y_pred = model.predict(X_test)
    predictions[name] = y_pred

    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

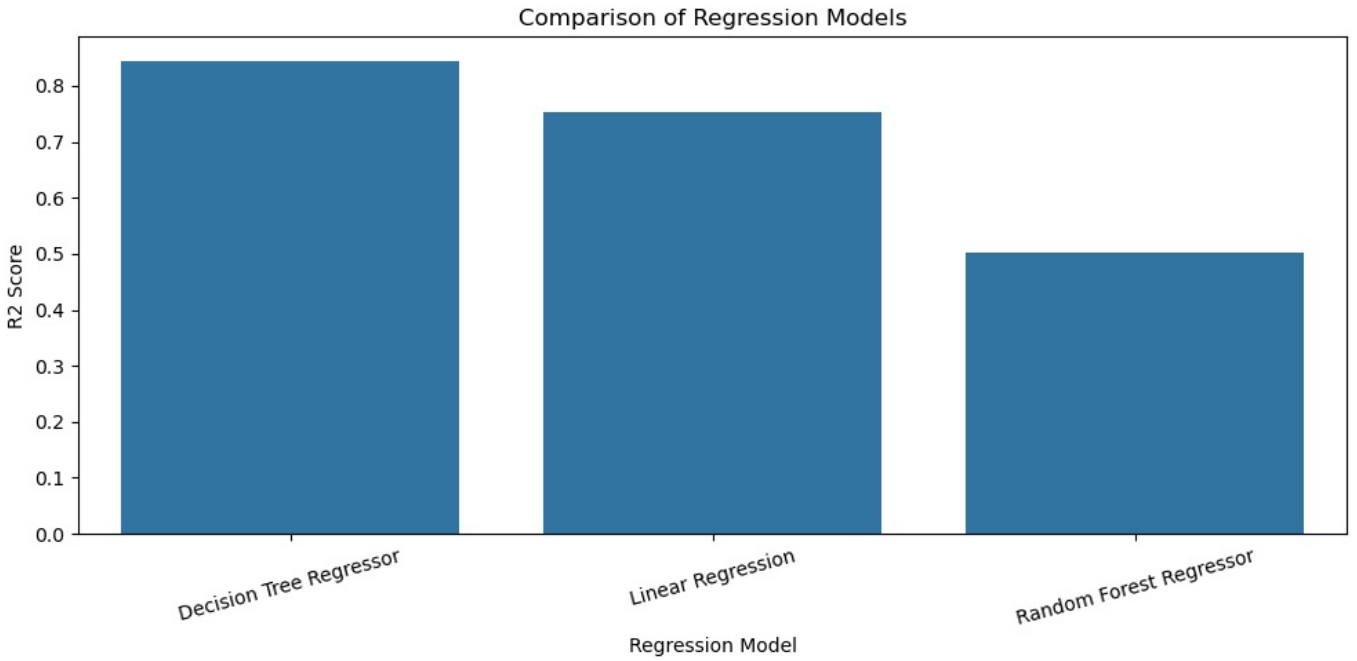
    results.append({
        "Model": name,
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2 Score": r2
    })
results_df = pd.DataFrame(results)
results_df = results_df.sort_values(
    by="R2 Score",
    ascending=False
).reset_index(drop=True)

print("Model Evaluation Results:")
display(results_df.round(4))
```

```
Model Evaluation Results:
```

	Model	MAE	MSE	RMSE	R2 Score
0	Decision Tree Regressor	1.0647	3.9882	1.9970	0.8453
1	Linear Regression	1.4729	6.3708	2.5240	0.7528
2	Random Forest Regressor	1.5007	12.8214	3.5807	0.5025

In [20]:
```
plt.figure(figsize=(10, 5))
sns.barplot(
    data=results_df,
    x="Model",
    y="R2 Score"
)
plt.title("Comparison of Regression Models")
plt.xlabel("Regression Model")
plt.ylabel("R2 Score")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```

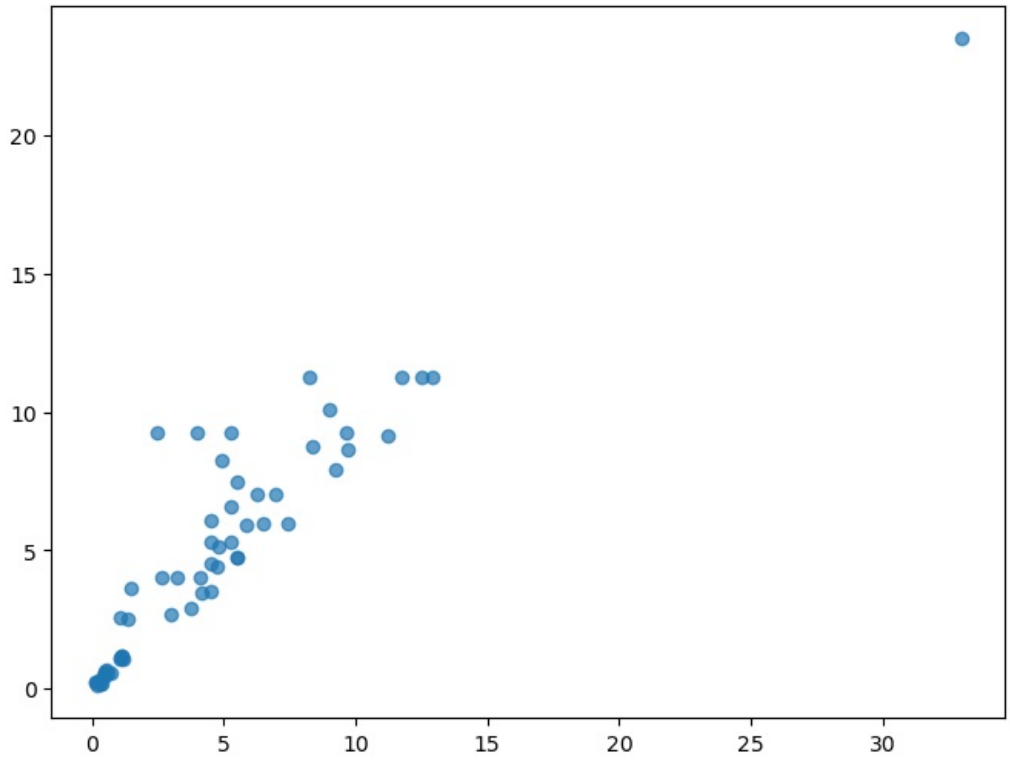


In [21]:
```
best_model_name = results_df.iloc[0]["Model"]
best_model = models[best_model_name]
print("Best Performing Model:", best_model_name)
print("MAE:", round(results_df.iloc[0]["MAE"], 4))
print("MSE:", round(results_df.iloc[0]["MSE"], 4))
print("RMSE:", round(results_df.iloc[0]["RMSE"], 4))
print("R2 Score:", round(results_df.iloc[0]["R2 Score"], 4))
```

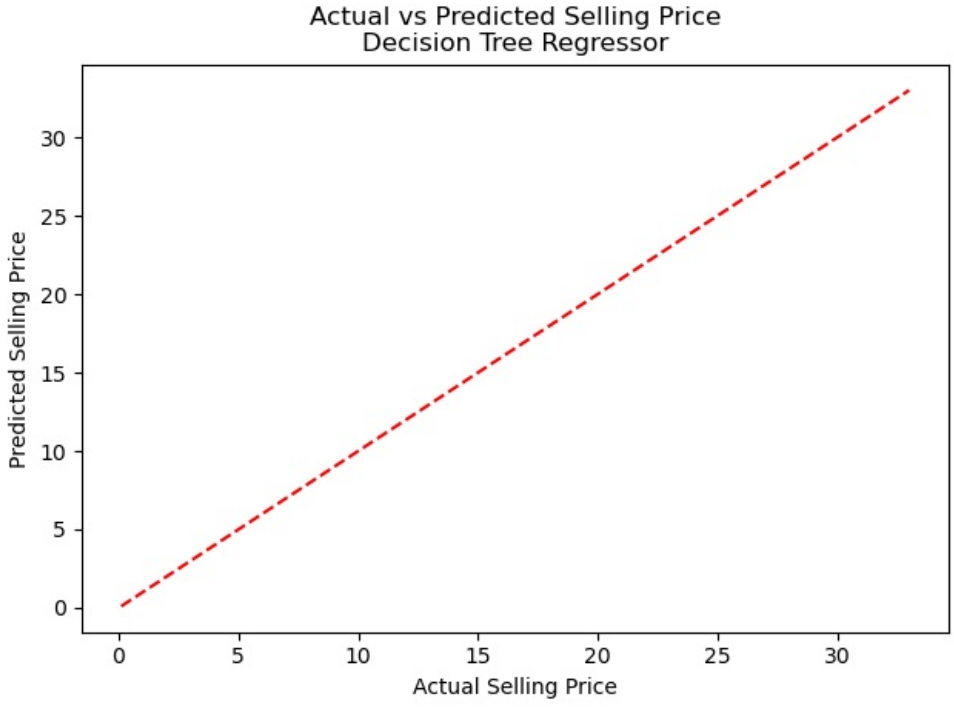
```
Best Performing Model: Decision Tree Regressor
MAE: 1.0647
MSE: 3.9882
RMSE: 1.997
R2 Score: 0.8453
```

In [22]:
```
best_predictions = predictions[best_model_name]
plt.figure(figsize=(8, 6))
plt.scatter(
    y_test,
    best_predictions,
    alpha=0.7
)
```

Out[22]:
```
<matplotlib.collections.PathCollection at 0x29ca0b84410>
```



In [23]:
```
min_value = min(y_test.min(), best_predictions.min())
max_value = max(y_test.max(), best_predictions.max())
plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    "r--"
)
plt.xlabel("Actual Selling Price")
plt.ylabel("Predicted Selling Price")
plt.title(f"Actual vs Predicted Selling Price\n{best_model_name}")
plt.tight_layout()
plt.show()
```



In [24]:
```
preprocessor = best_model.named_steps["preprocessor"]
trained_model = best_model.named_steps["model"]
feature_names = preprocessor.get_feature_names_out()
```

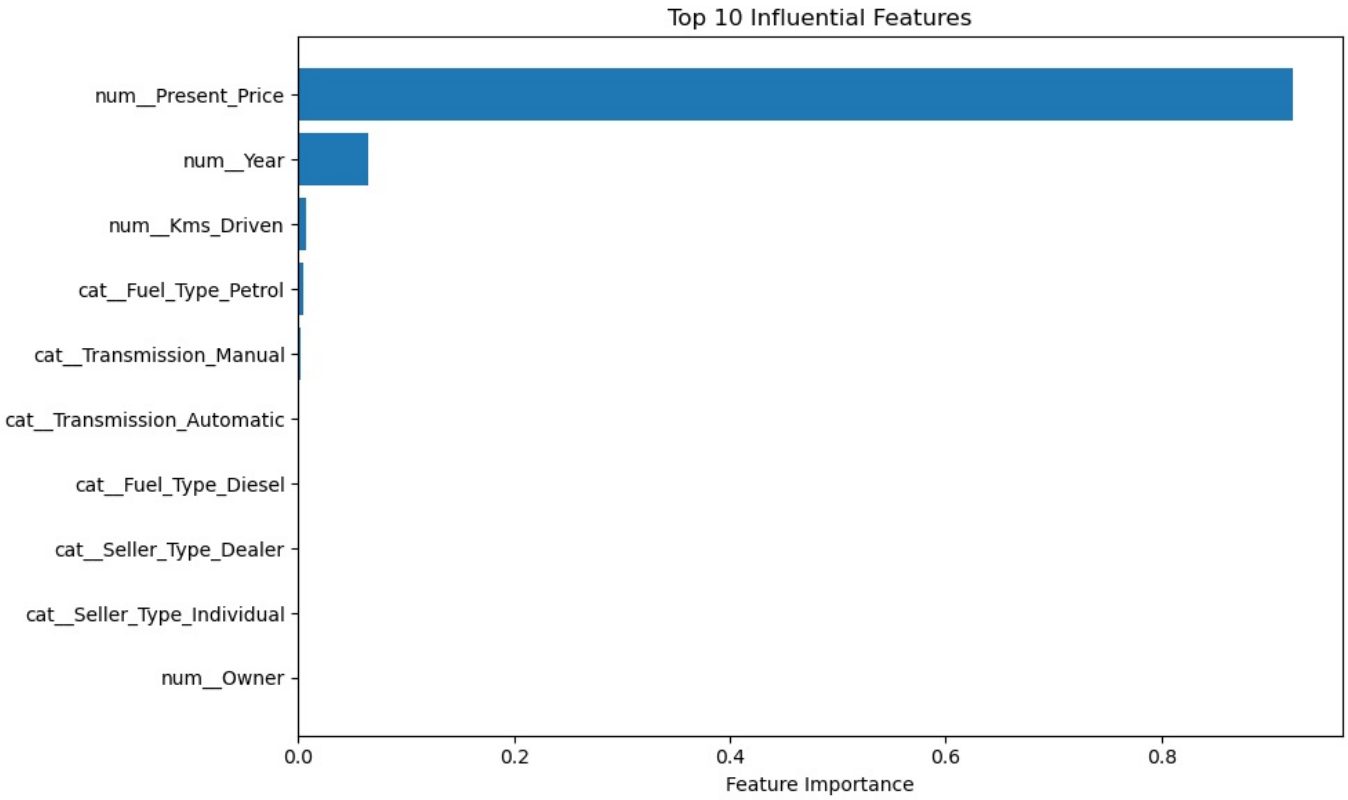
In [25]:
```
if hasattr(trained_model, "feature_importances_"):
    importance = trained_model.feature_importances_
    importance_label = "Feature Importance"
else:
    importance = np.abs(trained_model.coef_)
    importance_label = "Absolute Coefficient"

importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importance
})

importance_df = importance_df.sort_values(
    by="Importance",
    ascending=False
)
display(importance_df.head(10))
```

	Feature	Importance
1	num__Present_Price	9.219076e-01
0	num__Year	6.468383e-02
2	num__Kms_Driven	6.838668e-03
6	cat__Fuel_Type_Petrol	4.550604e-03
10	cat__Transmission_Manual	1.323623e-03
9	cat__Transmission_Automatic	2.935037e-04
5	cat__Fuel_Type_Diesel	2.036097e-04
7	cat__Seller_Type_Dealer	1.923732e-04
8	cat__Seller_Type_Individual	5.335796e-06
3	num__Owner	8.224241e-07

In [26]:
```
top_features = importance_df.head(10).sort_values(
    by="Importance",
    ascending=True
)
plt.figure(figsize=(10, 6))
plt.barh(
    top_features["Feature"],
    top_features["Importance"]
)
plt.xlabel(importance_label)
plt.title("Top 10 Influential Features")
plt.tight_layout()
plt.show()
```



In [27]:
```
best_row = results_df.iloc[0]
print("CONCLUSION")
print("-" * 50)
print(
    f"The best-performing model is {best_row['Model']} "
    f"based on the highest test R2 score."
)
print(f"MAE: {best_row['MAE']:.4f}")
print(f"MSE: {best_row['MSE']:.4f}")
print(f"RMSE: {best_row['RMSE']:.4f}")
print(f"R2 Score: {best_row['R2 Score']:.4f}")

print(
    "\nThe project compared Linear Regression, Decision Tree "
    "Regressor, and Random Forest Regressor for predicting "
    "used-car selling prices."
)
print(
    "The evaluation metrics and actual-versus-predicted "
    "visualization help assess the model's predictive performance."
)
```

```
CONCLUSION
--------------------------------------------------
The best-performing model is Decision Tree Regressor based on the highest test R2 score.
MAE: 1.0647
MSE: 3.9882
RMSE: 1.9970
R2 Score: 0.8453

The project compared Linear Regression, Decision Tree Regressor, and Random Forest Regressor for predicting used
-car selling prices.
The evaluation metrics and actual-versus-predicted visualization help assess the model's predictive performance.
```

In [ ]: